Engine version as a transfer axis (item 5)

Tier 2 found 12 of 22 queries get a different plan across PostgreSQL 14/15/17/18 on identical hardware and identical data, with PG15 ~30% slower than 17/18. **No cross-environment cost model in the literature treats engine version as an input.** This treats it exactly as the project treats hardware.

`s(q, A->B) = time(q,B) / time(q,A)`, evaluated **leave-one-version-out** so the target version is never seen in training — the direct analogue of leave-one-machine-out.

How big is the effect?

version pair	median scaling
15->17	0.708
15->18	0.740
15->14	0.796
18->17	0.934
18->14	0.947
14->17	0.967
17->14	1.035
14->18	1.056
17->18	1.070
14->15	1.256
18->15	1.353
17->15	1.413

Geometric-mean absolute runtime swing: **1.377x** for queries whose plan changed, **1.327x** for queries whose plan did not. So the version effect is real even when the plan is identical — it is not purely a planner story, it is also executor changes between major versions.

Can it be predicted for an unseen version?

predictor	MAPE %	MdAPE %	q-error (median)
naive	31.74	23.10	1.275
version_constant	31.74	23.10	1.275
plan_change	31.75	23.10	1.275
learned	31.74	23.10	1.275

Reading this

Ignoring the version entirely (`naive`) costs **31.74% MAPE**. The best predictor is `naive` at **31.74%**, an improvement of **+0.00pp**.

Nothing beats ignoring the version. With 4 versions, leave-one-version-out means the held-out version's *pairs* are also unseen, so a per-pair constant has nothing to fall back on but the global median. This is the n=5-machines problem in miniature, and it is the honest result: **version transfer is a real effect that this amount of data cannot model.** Four versions is to version-transfer what five machines is to hardware-transfer.

Per-pair constants are the interesting column regardless. The spread of median scaling across pairs is 0.708x to 1.413x — a factor of 2.00 between the most and least favourable version transition. A cost model that ignores this is absorbing it into its hardware term.

Why this matters for the paper

This is a **novel axis**, cheap to measure, and it undermines the hardware-signature abstraction from a second direction. The project's premise is `runtime = workload cost x hardware capability`. Version changes neither the workload nor the hardware, yet it moves runtime ~30% and changes half the plans. So the factorisation is incomplete, and the missing factor is not small.

Limitations

- **4 versions, one machine, 22 TPC-H queries.** Everything here is leave-one-of-four out.
- Each version ran at **its own defaults**, so part of the difference is changed defaults rather than changed planner or executor logic. Separating those needs each version run with the others' settings explicitly pinned — cheap, and the obvious next step.
- Statistics were rebuilt by `analyze` per version, so small cost differences may be statistics sampling rather than engine changes.